

Spring Transactions

Beginner-Friendly Guide with Examples, Diagrams & Q&As

Chapter 03 | Sub-Chapter 07

1. What is a Transaction?	ACID: Atomicity, Consistency, Isolation, Durability
2. @Transactional Basics	Declarative transaction management in Spring
3. Propagation Levels	REQUIRED, REQUIRES_NEW, NESTED, SUPPORTS, and more
4. Isolation Levels	READ_UNCOMMITTED to SERIALIZABLE
5. Rollback Rules	rollbackFor / noRollbackFor customization
6. Transaction Pitfalls	Self-invocation proxy bypass and gotchas
7. Programmatic Transactions	TransactionTemplate for fine-grained control

1 What is a Transaction?

ACID Properties Explained

A **transaction** is a sequence of operations treated as a single unit of work. Either all operations succeed (commit) or all fail (rollback). Spring leverages ACID to guarantee correctness.

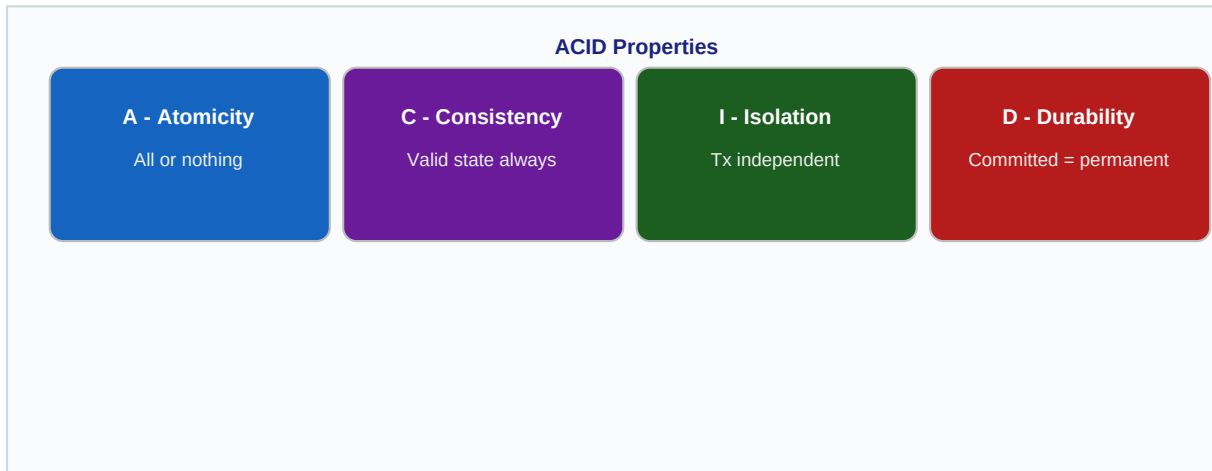


Figure 1.1 -- ACID: four pillars of a reliable transaction

Simple Java Example -- bank transfer:

```
@Service
public class BankService {
    @Transactional
    public void transfer(Long fromId, Long toId, BigDecimal amt) {
        Account from = repo.findById(fromId).orElseThrow();
        Account to = repo.findById(toId).orElseThrow();
        from.debit(amt);
        to.credit(amt);
        repo.save(from);
        repo.save(to);
    }
}
```

Key Rules

- Atomicity -- if step 2 fails, step 1 is rolled back automatically.
- Consistency -- account balances obey invariants before and after TX.
- Isolation -- concurrent transfers do not see each other partial work.
- Durability -- once committed, data survives a crash.

Q1. What is a transaction?

A unit of work where all steps succeed together or all are undone together.

Q2. What does Atomicity mean?

All operations in the transaction complete, or none of them are applied.

Q3. What does Isolation mean?

Each transaction executes as if it were the only one running concurrently.

Q4. What happens if an exception is thrown mid-transaction?

Spring rolls back all changes made during that transaction automatically.

Q5. Why are transactions important in banking apps?

They prevent partial updates (e.g., money deducted but not credited) that corrupt data.

2 @Transactional Annotation Basics

Declarative Transaction Management

Spring's **@Transactional** annotation lets you declare transaction boundaries without writing JDBC boilerplate. Spring creates a proxy that intercepts the method call, starts a transaction, executes the method, then commits or rolls back.

```
@Configuration
@EnableTransactionManagement
public class AppConfig { }

@Service
public class OrderService {
    @Autowired OrderRepository repo;

    @Transactional
    public void placeOrder(Order order) {
        repo.save(order);
        inventoryService.reduce(order.getItems());
    }
}
```

Key Rules

- **@Transactional** on a class applies to all public methods.
- Only public methods on Spring beans are intercepted by the proxy.
- Defaults: propagation=REQUIRED, isolation=DEFAULT, rollbackOn=RuntimeException.
- Use **@EnableTransactionManagement** in **@Configuration** to activate support.

Q1. What does **@Transactional** do?

It wraps the method in a transaction: begin before the call, commit after success, rollback on exception.

Q2. Where should **@Transactional** be placed?

On the service layer to keep business logic boundaries clear.

Q3. Does **@Transactional** work on private methods?

No. Spring proxy only intercepts public methods called from outside the bean.

Q4. What exception type triggers automatic rollback by default?

RuntimeException and its subclasses. Checked exceptions do NOT roll back by default.

Q5. How do you enable **@Transactional** support in Spring Boot?

Spring Boot auto-configures it. In plain Spring add **@EnableTransactionManagement** to a **@Configuration** class.

3 Propagation Levels

Controlling How Transactions Spread

Propagation defines what happens when a `@Transactional` method is called while a transaction already exists (or does not exist).

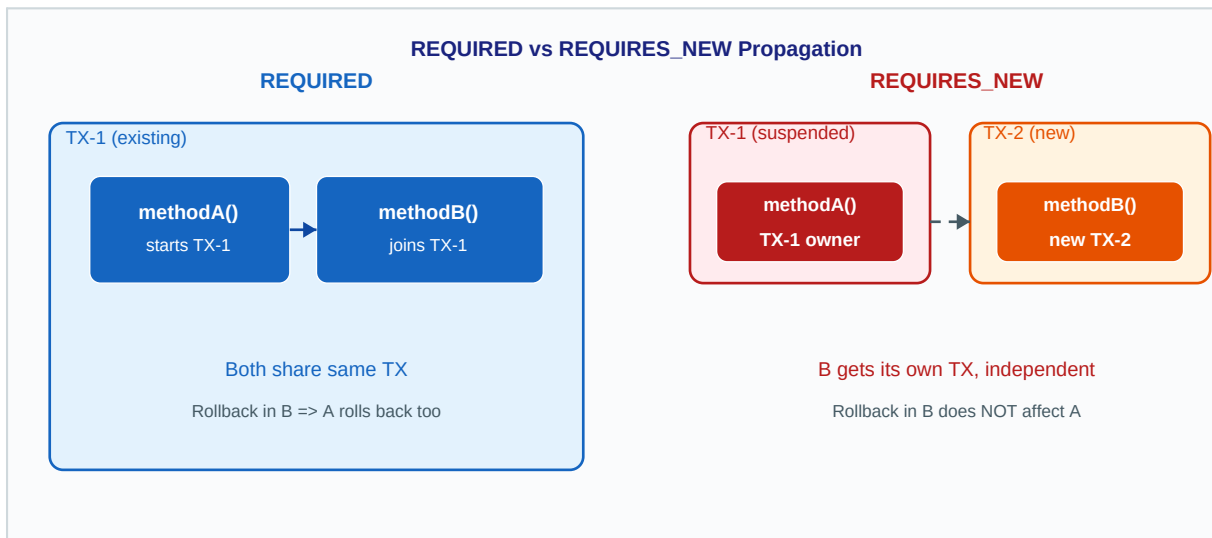


Figure 3.1 -- REQUIRED reuses the existing TX; REQUIRES_NEW suspends it and starts fresh

```
@Transactional(propagation = Propagation.REQUIRED)
public void methodA() { } // join existing or create new

@Transactional(propagation = Propagation.REQUIRES_NEW)
public void methodB() { } // always a fresh TX

@Transactional(propagation = Propagation.NESTED)
public void methodC() { } // savepoint inside outer TX

@Transactional(propagation = Propagation.SUPPORTS)
public void methodD() { } // TX if present, else none

@Transactional(propagation = Propagation.NOT_SUPPORTED)
public void methodE() { } // suspend TX, run outside

@Transactional(propagation = Propagation.MANDATORY)
public void methodF() { } // throw if no TX exists

@Transactional(propagation = Propagation.NEVER)
public void methodG() { } // throw if TX exists
```

Key Rules

- REQUIRED (default): join existing TX or create new one.
- REQUIRES_NEW: always create a new TX; suspend the caller TX.
- NESTED: create a savepoint inside the current TX (JDBC only).
- SUPPORTS: run transactionally only if a TX already exists.
- NOT_SUPPORTED: suspend any existing TX; run outside a TX.
- MANDATORY: throw if called without an active TX.
- NEVER: throw if called inside an active TX.

Q1. What is the default propagation level?

REQUIRED -- the method joins an existing transaction or starts a new one if none exists.

Q2. When would you use REQUIRES_NEW?

When you need the inner operation (e.g., audit log) to commit independently of the outer TX.

Q3. What does NESTED propagation use internally?

JDBC savepoints, so it only works with JDBC-based transaction managers.

Q4. What exception does MANDATORY throw if no TX is active?

IllegalTransactionStateException -- indicating a programming error.

Q5. How is SUPPORTS different from REQUIRED?

SUPPORTS does not start a new TX; it just participates if one already exists.

4 Isolation Levels

Controlling Concurrent Transaction Visibility

Isolation level controls how much one transaction can see the uncommitted changes of another. Higher isolation = more consistent reads, but lower concurrency.

```
@Transactional(isolation = Isolation.READ_UNCOMMITTED)
public void method1() { } // fastest, dirty reads possible

@Transactional(isolation = Isolation.READ_COMMITTED)
public void method2() { } // default in most DBs

@Transactional(isolation = Isolation.REPEATABLE_READ)
public void method3() { } // same query = same result

@Transactional(isolation = Isolation.SERIALIZABLE)
public void method4() { } // strictest, full isolation
```

Level	Dirty Read	Non-Repeatable Read	Phantom Read	Use When
READ_UNCOMMITTED	Yes	Yes	Yes	Fastest reads, stale data OK
READ_COMMITTED	No	Yes	Yes	Most apps (Postgres default)
REPEATABLE_READ	No	No	Yes	Reports needing stable data
SERIALIZABLE	No	No	No	Financial: max correctness

Key Rules

- Dirty Read: reading uncommitted data from another TX.
- Non-Repeatable Read: same row returns different values within same TX.
- Phantom Read: same query returns different rows within same TX.
- DEFAULT lets the DB choose (usually READ_COMMITTED).
- Higher isolation = more locking = lower throughput.

Q1. What is a dirty read?

Reading data written by a transaction that has not yet committed -- the data may be rolled back.

Q2. Which isolation level prevents dirty reads but allows non-repeatable reads?

READ_COMMITTED -- the default in PostgreSQL and SQL Server.

Q3. What problem does REPEATABLE_READ solve?

It guarantees that re-reading the same row within a TX always returns the same value.

Q4. When should you use SERIALIZABLE?

For financial calculations where phantom reads must be prevented, accepting reduced concurrency.

Q5. What is Isolation.DEFAULT?

It delegates to the underlying database default isolation level.

5 Rollback Rules

rollbackFor and noRollbackFor

By default Spring rolls back on **unchecked exceptions (RuntimeException)** and **Errors**, but NOT on checked exceptions. Use rollbackFor / noRollbackFor to customise this.

```
// Force rollback for a checked exception
@Transactional(rollbackFor = IOException.class)
public void saveFile(File f) throws IOException {
    repo.save(f);
    fileSystem.write(f); // IOException here => rollback
}

// Prevent rollback for a specific RuntimeException
@Transactional(noRollbackFor = OptimisticLockException.class)
public void update(Entity e) {
    repo.save(e); // OptimisticLockException => still commit
}
```

Key Rules

- Default rollback: RuntimeException and Error subclasses.
- Checked exceptions do NOT trigger rollback unless in rollbackFor.
- rollbackFor accepts one or more exception classes.
- noRollbackFor lets you commit even when the listed exception is thrown.
- Can combine: rollbackFor = Exception.class, noRollbackFor = MyIgnored.class.

Q1. Does @Transactional roll back on checked exceptions by default?

No. You must explicitly declare rollbackFor = YourCheckedException.class.

Q2. What is the syntax to roll back on any exception?

@Transactional(rollbackFor = Exception.class) catches all checked and unchecked.

Q3. What does noRollbackFor do?

It suppresses rollback for the listed exceptions, allowing the TX to commit.

Q4. If a method throws FileNotFoundException, does it roll back by default?

No -- FileNotFoundException is checked. Add rollbackFor = FileNotFoundException.class.

Q5. Can you specify multiple exceptions in rollbackFor?

Yes: rollbackFor = {IOException.class, SQLException.class}.

6 Transaction Pitfalls

Self-Invocation and Other Gotchas

The most common Spring transaction bug is **self-invocation**: calling a `@Transactional` method from within the same bean bypasses the proxy, so the annotation has no effect.

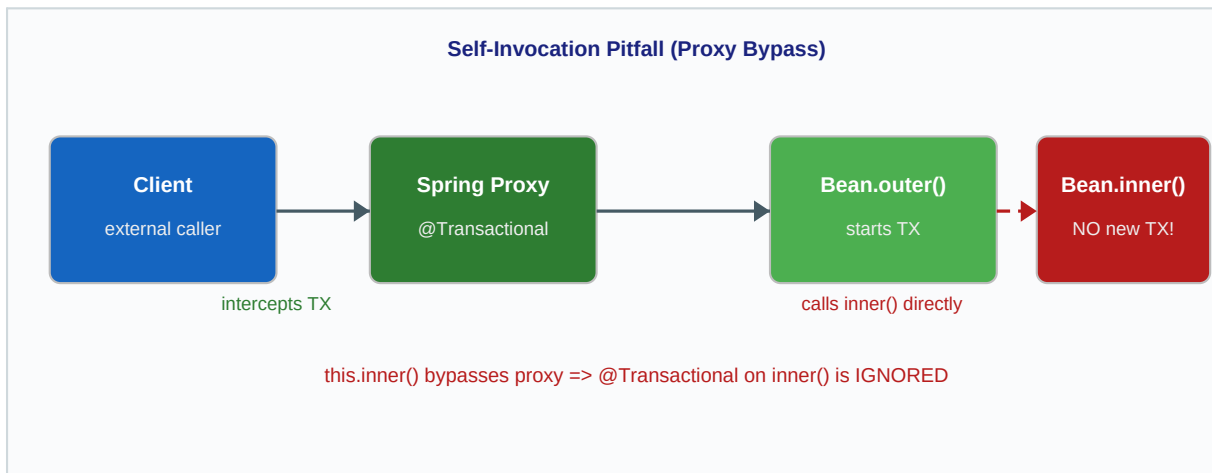


Figure 6.1 -- this.inner() skips the proxy; @Transactional on inner() is silently ignored

```
@Service
public class OrderService {
    // WRONG: this.inner() bypasses proxy -- no TX started
    public void outer() {
        this.inner();
    }
    @Transactional
    public void inner() { /* TX never started! */ }
}

// FIX: inject self, call via proxy
@Autowired private OrderService self;
public void outer() {
    self.inner(); // proxy intercepts -- TX starts correctly
}
```

Key Rules

- Spring `@Transactional` uses a proxy -- only external calls are intercepted.
- `this.method()` inside the same bean bypasses the proxy -- no TX.
- Fix 1: inject the bean into itself and call via `self.method()`.
- Fix 2: move the transactional method to a separate Spring bean.
- Checked exceptions do NOT roll back by default -- a silent gotcha.
- `@Transactional` on private or final methods has no effect.

Q1. Why does self-invocation bypass @Transactional?

Spring wraps beans in a proxy. Calls via `this` skip the proxy, so no transaction advice is applied.

Q2. How do you fix a self-invocation problem?

Inject the bean itself (self-injection) or extract the method into a separate Spring-managed bean.

Q3. If a checked exception escapes a @Transactional method, does it roll back?

No -- unless you add `rollbackFor = CheckedException.class` to the annotation.

Q4. Does @Transactional work on private methods?

No. The proxy only intercepts public methods; private methods are called directly.

Q5. What happens if you annotate a final method with @Transactional?

CGLib proxies cannot override final methods, so the annotation is silently ignored.

7 Programmatic Transactions

TransactionTemplate for Fine-Grained Control

When you need transaction boundaries inside a method -- not around the whole method -- use **TransactionTemplate** for explicit, programmatic control.

```
@Service
public class ReportService {
    @Autowired TransactionTemplate txTemplate;

    public void generateReport() {
        // runs inside a transaction
        txTemplate.execute(status -> {
            repo.save(newRecord());
            return null; // commit
        });
        sendEmail(); // outside any transaction
    }
}

@Bean
public TransactionTemplate txTemplate(PlatformTransactionManager tm) {
    return new TransactionTemplate(tm);
}
```

Key Rules

- TransactionTemplate handles begin/commit/rollback automatically.
- Return null from execute() to commit; throw RuntimeException to rollback.
- Call status.setRollbackOnly() to rollback without throwing an exception.
- Useful when only part of a method needs a transaction boundary.
- PlatformTransactionManager is provided by Spring Boot auto-configuration.
- TransactionCallback is the functional interface passed to execute().

Q1. When should you use TransactionTemplate instead of @Transactional?

When you need transaction boundaries mid-method, or in lambdas where annotations cannot be placed.

Q2. How do you trigger a rollback in TransactionTemplate without throwing?

Call status.setRollbackOnly() inside the execute() callback.

Q3. What does TransactionTemplate.execute() return?

Whatever you return from the TransactionCallback lambda, or null.

Q4. Can you set propagation on TransactionTemplate?

Yes: txTemplate.setPropagationBehavior(TransactionDefinition.PROPGATION_REQUIRES_NEW).

Q5. What is PlatformTransactionManager?

Spring central interface for TX management. DataSourceTransactionManager and JpaTransactionManager are common implementations.